

STATIC WEBSITE HOSTING ON AWS EC2 USING NGINX

Mini Project Report

Submitted in partial fulfillment of the requirements for the course

Cloud Computing

Submitted by

D. Prince Joyal Dhas

S. No.	Contents	Page No.
1	Abstract	3
2	Introduction	3
3	Objectives	4
4	System Requirements	4
5	Tools and Technologies Used	5
6	Implementation	5
7	Commands Used	7
8	Testing	7
9	Results	7
10	Challenges Faced	8
11	Conclusion	8
12	Future Enhancements	8
13	References	9

ABSTRACT

This project demonstrates the deployment of a static portfolio website using Amazon Web Services (AWS). An Amazon EC2 instance running Ubuntu Server was launched and configured as the hosting environment. Nginx was installed and configured as the web server to host the website. The website was developed using HTML and CSS and securely transferred to the EC2 instance using Secure Copy Protocol (SCP). Secure Shell (SSH) was used for remote server administration. The deployed website was successfully accessed through the public IPv4 address of the EC2 instance. This project provided practical experience in cloud computing, Linux server management, web hosting, and deployment of web applications on AWS.

INTRODUCTION

Cloud computing enables users to deploy applications without investing in physical infrastructure. Amazon Web Services (AWS) provides scalable and reliable cloud services for hosting applications. One of the most widely used services is Amazon Elastic Compute Cloud (EC2), which provides virtual servers for deploying applications. In this project, a static portfolio website was hosted on an Ubuntu EC2 instance using the Nginx web server. The project demonstrates the complete deployment process, including server creation, security configuration, website deployment, and remote server management.

OBJECTIVES

- To launch and configure an Amazon EC2 instance.
- To establish secure remote access using SSH.
- To install and configure the Nginx web server.
- To develop a static portfolio website using HTML and CSS.
- To deploy the website on an EC2 instance.
- To verify successful hosting using the public IPv4 address.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB or higher
- Storage: 20 GB free disk space
- Internet Connection

Software Requirements

- Windows 10/11
- AWS Management Console
- Ubuntu Server 24.04 LTS
- Nginx Web Server
- HTML5
- CSS3
- OpenSSH

- Google Chrome / Microsoft Edge
- Visual Studio Code

TOOLS AND TECHNOLOGIES USED

Tool / Technology	Purpose
AWS EC2	Virtual server for hosting the website
Ubuntu Server	Operating system running on EC2
Nginx	Web server used to host the website
HTML5	Structure of the website
CSS3	Styling of the website
SSH	Secure remote login to EC2
SCP	Secure file transfer from local system to EC2
Visual Studio Code	HTML and CSS development
Web Browser	Accessing and testing the website

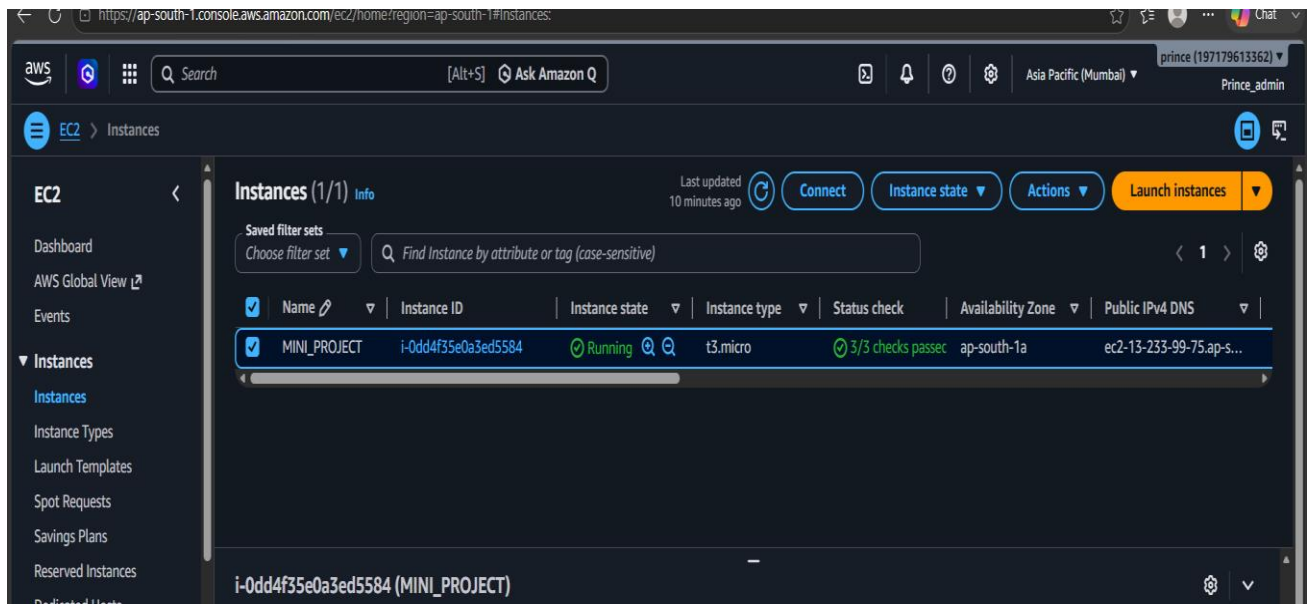
PROJECT ARCHITECTURE

The project follows a simple cloud-based web hosting architecture. The user accesses the website using a web browser. The request is sent through the Internet to the public IPv4 address of the Amazon EC2 instance. The Security Group allows HTTP traffic on Port 80 and SSH traffic on Port 22. The Ubuntu operating system hosts the Nginx web server, which retrieves the HTML and CSS files stored in the `/var/www/html` directory and serves them to the user's browser.

IMPLEMENTATION

Launching the EC2 Instance

An Ubuntu EC2 instance was launched in the AWS Management Console using the default VPC. A security group was configured to allow SSH (Port 22) and HTTP (Port 80). A key pair was generated for secure authentication.



Connecting to the Server

The EC2 instance was accessed remotely using SSH with the downloaded PEM key file. After successful authentication, administrative tasks were performed through the Linux terminal.

Installing Nginx

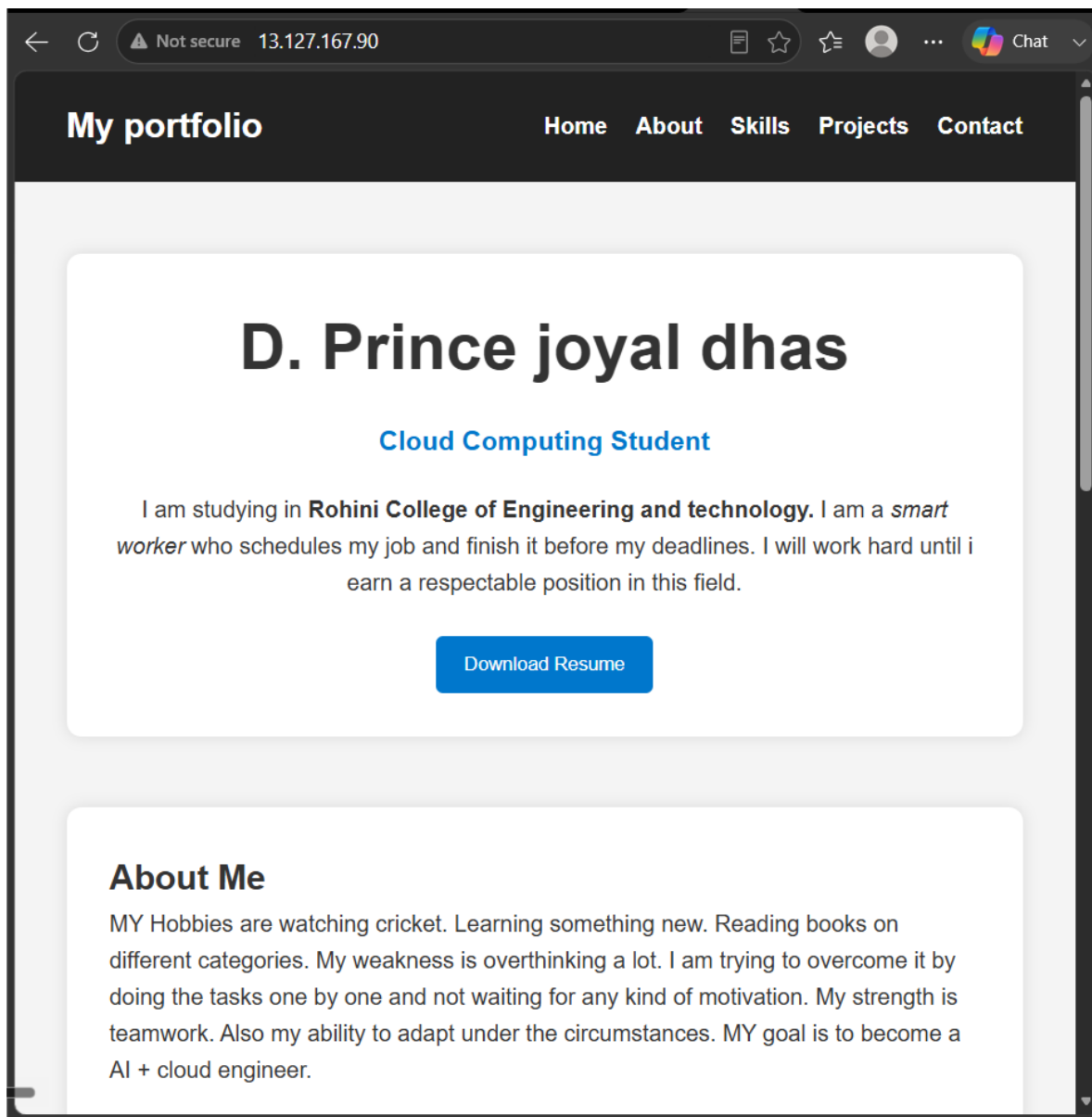
The package repository was updated and the Nginx web server was installed using the APT package manager. The Nginx service was verified to ensure it was running successfully.

Developing the Website

A static portfolio website was developed using HTML and CSS. The website contains navigation, personal information, skills, project details, and contact information.

Deploying the Website

The website files were securely transferred from the local computer to the EC2 instance using SCP. The HTML and CSS files were copied into the `/var/www/html` directory, replacing the default Nginx page. The deployed website was successfully accessed using the EC2 public IPv4 address.



COMMANDS USED

Command	Purpose
ssh -i MY_EC2_KEY2.pem ubuntu@<Public-IP>	Connect to EC2 instance
sudo apt update	Update package list
sudo apt install nginx -y	Install Nginx
sudo systemctl status nginx	Check Nginx status
scp -i MY_EC2_KEY2.pem -r "Website Files" ubuntu@<Public-IP>:/home/ubuntu	Copy website files to EC2
sudo cp index.html /var/www/html/	Copy HTML file to Nginx directory
sudo cp style.css /var/www/html/	Copy CSS file to Nginx directory
ls -l /var/www/html	Verify website files

TESTING

Test Case	Expected Result	Actual Result	Status
SSH Connection	Successful login to Ubuntu server	Successfully connected	Pass
Nginx Installation	Nginx service running	Service active (running)	Pass
Website Deployment	HTML and CSS files copied successfully	Files copied successfully	Pass
Website Access	Website loads through Public IPv4 address	Website displayed successfully	Pass

RESULTS

The static portfolio website was successfully deployed on an Amazon EC2 Ubuntu instance using the Nginx web server. The website was accessible through the EC2 public IPv4 address over

the Internet. Secure remote administration was achieved using SSH, and website files were transferred securely using SCP. The project demonstrated successful deployment of a cloud-hosted static website.

```
ubuntu@ip-172-31-37-245: ~  
Preparing to unpack .../nginx_1.28.3-2ubuntu1.6_amd64v3.deb ...  
Unpacking nginx (1.28.3-2ubuntu1.6) ...  
Setting up nginx-common (1.28.3-2ubuntu1.6) ...  
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service'  
→ '/usr/lib/systemd/system/nginx.service'.  
Setting up nginx (1.28.3-2ubuntu1.6) ...  
* Upgrading binary nginx [ OK ]  
Processing triggers for man-db (2.13.1-1build1) ...  
Processing triggers for ufw (0.36.2-9build1) ...  
Scanning processes...  
Scanning linux images...  
  
Running kernel seems to be up-to-date.  
  
No services need to be restarted.  
  
No containers need to be restarted.  
  
No user sessions are running outdated binaries.  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@ip-172-31-37-245:~$ sudo systemctl status nginx  
● nginx.service - A high performance web server and a reverse proxy server  
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset>  
   Active: active (running) since Sat 2026-07-11 09:39:05 UTC; 1min 35s a>  
 Invocation: 66a7fb5f71474da4bcda3421ad284e15  
    Docs: man:nginx(8)  
  Process: 1791 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_p>  
  Process: 1793 ExecStart=/usr/sbin/nginx -g daemon on; master_process on>  
 Main PID: 1822 (nginx)  
   Tasks: 3 (limit: 627)  
  Memory: 3M (peak: 7.3M)  
    CPU: 42ms  
   CGroup: /system.slice/nginx.service  
           └─1822 "nginx: master process /usr/sbin/nginx -g daemon on; ma>  
             └─1825 "nginx: worker process"  
             └─1826 "nginx: worker process"  
  
Jul 11 09:39:05 ip-172-31-37-245 systemd[1]: Starting nginx.service - A hig>  
Jul 11 09:39:05 ip-172-31-37-245 systemd[1]: Started nginx.service - A high>  
ubuntu@ip-172-31-37-245:~$ sudo apt update
```

CHALLENGES FACED

- Initial SSH connection timed out due to network configuration.
- Understanding AWS Security Groups and SSH access.
- Configuring the Nginx web server.
- Deploying website files to the correct directory.
- Managing EC2 public IP changes after restarting the instance.

CONCLUSION

This project provided practical knowledge of cloud computing by deploying a static website on Amazon EC2. The project improved understanding of Linux administration, web server configuration, secure remote access, file deployment, and AWS cloud infrastructure. The successful deployment demonstrated the practical implementation of website hosting using cloud services.

FUTURE ENHANCEMENTS

- Deploy the website using HTTPS with SSL/TLS certificates.
- Register and connect a custom domain name.
- Improve the website design using JavaScript and modern CSS.
- Automate deployment using CI/CD pipelines.

- Migrate the website to a serverless hosting platform such as Amazon S3 and CloudFront.

REFERENCES

1. AWS EC2 Documentation
2. Ubuntu Server Documentation
3. Nginx Documentation
4. HTML5 Documentation (MDN Web Docs)
5. CSS3 Documentation (MDN Web Docs)